PAI: A Novel Debt-Native Blockchain Protocol

PAI Protocol

October 30, 2026

Abstract

PAI is a debt-native blockchain protocol that introduces a novel lending primitive built around non-transferable-in-default debt instruments. The protocol enables deposits of external assets to create debt-tracked instruments, generates loanable currencies derived from those deposits, and enforces strict debt obligations through immutable records. A custom consensus mechanism called *Custom Attested Bundle* coordinates block production through sealed-bid auctions between paired producers and a rotating reputation-weighted committee. The system employs block-relevant validation against a shared replicated ledger, enforces one-transaction full debt settlement on instrument transfers with automatic burning on failure, and implements liquidation through pools with arbitrage mechanisms. The native currency PAI has a dynamic supply with fee burning and rewards distributed to network participants. The protocol further introduces a bounty lifecycle: when a depositor withdraws before unpaid loans are settled, a bounty is created from the withdrawn value and auctioned on-chain, with settlement to the highest bidder or, absent bids, proportional distribution and timed burning. In addition, when a borrower fully repays a debt instrument with no prior repayments, that borrower becomes a yield splitter: a portion of the interest that would otherwise accrue to depositors is converted one-to-one to PAI and accrued as splitter earnings, and every six repayments each splitter distributes ten percent of accrued earnings to the original depositor while receiving taker-score units. Depositors may freeze an instrument to govern the minimum splitter count and yield splitters vote on the split percentage before the instrument unfreezes. This paper presents the complete system design, economic mechanisms, and security considerations.

Contents

1 Introduction **2**

2 System Overview **2**

3 Network Architecture **2**
- 3.1 Node Roles . . . 2
- 3.2 Shared Replicated Ledger . . . 3
- 3.3 Block-Relevant Validation . . . 3

4 Consensus: Custom Attested Bundle **3**
- 4.1 Sealed-Bid Auction . . . 3
- 4.2 Pair Selection . . . 3
- 4.3 Committee Formation . . . 3
- 4.4 Attestation and Quorum . . . 3

5 Asset Deposits & Debt Instruments **4**
- 5.1 Deposit Mechanism . . . 4
- 5.2 Debt Instruments . . . 4

6 Lending & Interest **4**

7 Liquidation & Arbitrage **5**
7.1 Liquidation Triggers . . . 5
7.2 Liquidation Pools . . . 5
7.3 Arbitrage Mechanism . . . 5

8 Debt Instruments & Trading **5**
8.1 Transfer Restrictions . . . 5
8.2 One-Transaction Settlement Requirement . . . 6
8.3 Loanability Reset on Successful Transfer . . . 6
8.4 Burning Mechanism . . . 6
8.5 Profit Distribution on Sale . . . 6

9 Value Factor Mechanics **7**
9.1 Definition . . . 7
9.2 Behavior . . . 7

10 Depositor Withdrawal **8**
10.1 Overview . . . 8
10.2 Withdrawal Execution . . . 8
10.3 Affected Loan Adjustment . . . 8
10.4 Withdrawn Funds Tracking and Loss . . . 8
10.5 Bounty Hook . . . 8

11 Yield Splitting for Fully Repaid Instruments **8**
11.1 Overview . . . 8
11.2 Qualifying Repayment . . . 9
11.3 Yield Splitter Addition and Initial PAI Payment . . . 9
11.4 Recurring Distribution . . . 9
11.5 Freeze Enforcement . . . 10
11.6 Depositor Freeze and Minimum Splitter Count . . . 10
11.7 Yield Splitter Voting and Establishment . . . 10
11.8 Data Model . . . 10
11.9 Protocol Constants . . . 11

12 Bounty System **11**
12.1 Overview . . . 11
12.2 Bounty Creation . . . 11
12.3 Loanable Reduction and Payment Schedules . . . 11
12.4 Consistency Scoring and Time Limits . . . 12
12.5 On-Chain Auction and Bidding . . . 12
12.6 Expiry Processing . . . 12
12.7 Timed Burn . . . 12
12.8 Economic Invariants . . . 12

13 Bridging **13**

14 PAI Economic Model **13**
14.1 Native Currency . . . 13
14.2 Supply Mechanics . . . 13
14.3 Fee Market . . . 13
14.4 Reward Distribution . . . 14

15 Validation, Attestation & Cross-Verification **14**
15.1 Block-Relevant State Construction 14
15.2 Attestation Process 14
15.3 Cross-Verification 14

16 Reputation & Slashing **14**
16.1 Reputation System 14
16.2 Slashing 15
16.3 Committee Selection 15

17 Security Considerations **15**
17.1 Economic Security 15
17.2 Debt Enforcement 15
17.3 Consensus Security 16
17.4 Invariant Enforcement 16

18 Conclusion **16**

19 References **16**

20 Appendix **17**
20.1 Key Parameters 17
20.2 Glossary 17

1 Introduction

Traditional blockchain systems lack native primitives for debt creation, enforcement, and liquidation in a unified manner. PAI addresses this gap by treating debt as a first-class primitive embedded at the protocol level. Rather than relying solely on smart contracts for lending logic, PAI encodes debt obligations, value mechanics, and liquidation paths directly into the protocol state machine.

The protocol introduces several innovations: (1) deposit-derived debt instruments with immutable history, (2) a custom consensus model combining pairwise production with committee attestation, (3) block-relevant validation that maintains security while limiting computational scope, (4) strict transfer mechanics requiring full debt settlement in a single transaction, (5) automatic liquidation escalation through pools to arbitrageurs, and (6) a ratio-based value factor mechanism, and (7) a bounty lifecycle that auctions withdrawn value when loans remain unpaid, and (8) yield splitting that rewards borrowers who fully repay with no prior repayments by diverting a portion of depositor interest to them as PAI, with recurring distributions and depositor-governed freeze/vote controls.

2 System Overview

PAI operates as a distributed system where network participants maintain identical copies of a shared replicated immutable ledger. All state transitions are append-only and fully auditable through immutable history records. The protocol enforces economic invariants at the consensus and execution layers, ensuring that debt obligations cannot be circumvented and that failed settlements trigger predetermined consequences.

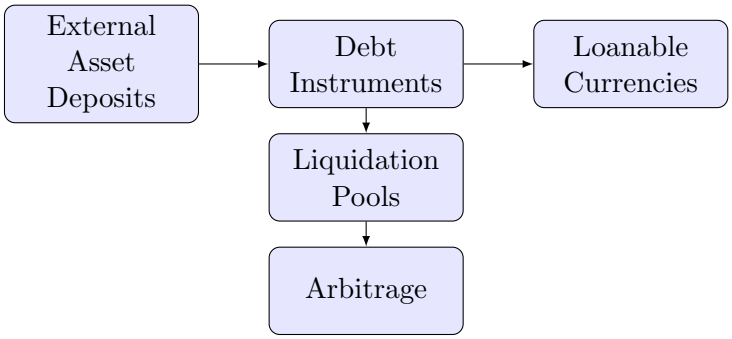


Figure 1: Core economic flow in PAI

3 Network Architecture

3.1 Node Roles

PAI distinguishes between two primary node roles:

- **Block Producers**: Collect transactions, assemble blocks, maintain the shared replicated ledger, and verify attestations from validators.
- **Validators**: Verify transaction validity against block-relevant state, produce attestations, and maintain identical copies of the shared replicated ledger.

Both roles maintain complete copies of the shared replicated immutable ledger. However, validation is performed only on transactions relevant to the current block, enabling efficient operation without requiring full-chain reprocessing.

3.2 Shared Replicated Ledger

The shared replicated ledger serves as the canonical source of truth maintained identically across all network participants. Properties include:

- Append-only immutability
- Complete transaction and state history
- Block-relevant indexing for efficient validation
- Cryptographic commitments over state

3.3 Block-Relevant Validation

Rather than validating all historical state, nodes construct per-block state views containing only touched accounts, assets, and instruments. This approach maintains security guarantees while reducing validation scope to the minimal set required for correctness.

4 Consensus: Custom Attested Bundle

PAI employs *Custom Attested Bundle* consensus, a hybrid model combining auction-based selection, pairwise production, and committee attestation.

4.1 Sealed-Bid Auction

For each block, eligible participants submit sealed bids to secure production rights. The auction proceeds in two phases:

1. **Commit Phase**: Participants submit commitments $C_i = H(b_i \| \eta_i)$ where b_i is the bid amount in PAI and η_i is a random nonce.
2. **Reveal Phase**: Participants reveal (b_i, η_i), allowing verification against commitments.

4.2 Pair Selection

From revealed bids, the highest bidder is selected as the block producer, and a second distinct participant is selected as the paired validator. A participant cannot be paired with themselves. The winning bid amount is paid if selected.

4.3 Committee Formation

A rotating committee of fixed size $k = 4$ is selected from remaining eligible participants using reputation-weighted selection. The committee excludes members of the active pair, preventing collusion between producers and validators.

4.4 Attestation and Quorum

The paired validator produces an attestation of block validity after performing block-relevant validation against the shared replicated ledger. The block producer verifies this attestation against the same ledger state. Finalization requires quorum from the pair and majority support from the committee.



Figure 2: Custom Attested Bundle consensus flow

5 Asset Deposits & Debt Instruments

5.1 Deposit Mechanism

Users deposit any external bridged asset into the protocol. Upon deposit confirmation, the protocol creates a debt-tracked instrument linked to the deposit. The deposit amount is split according to consensus parameters into three components:

- Loanable portion for currency creation
- Reserve/backup portion
- Interest pool portion

The loanable portion is computed from the deposit and the instrument's current loan count:

$$\text{loanable_pct} = \text{clamp}(70 - 5 \cdot \text{loan_count},\ 20,\ 70) \tag{1}$$

$$\text{loanable} = \text{deposit_amount} \cdot \frac{\text{loanable_pct}}{100} \tag{2}$$

This value is stored both as `total_minted_from_nft` (the denominator used by the value factor) and as `deposit_loanability`. At deposit time `loan_count` is zero, so the initial loanability is 70% of the deposit.

5.2 Debt Instruments

Each deposit generates a unique debt instrument with the following properties:

- Immutable history of all events (deposits, loans, repayments, transfers, liquidations)
- Collateral backing from the original deposit
- Associated loanable currency
- Tracked debt balance and interest obligations
- Value factor determining economic weight
- Deposit loanability (`deposit_loanability`) recording the deposit-derived loanable amount
- Loan count (`loan_count`) tracking loans associated with the instrument
- Status indicating active, liquidating, burned, or paid-off state

The instrument history is append-only and cannot be modified, ensuring complete auditability of all debt-related events.

6 Lending & Interest

Loanable currencies are created from deposits and can be issued against debt instruments. Interest accrues according to the rate established at instrument creation. Repayments must be made in the original currency context, with interest calculated as:

$$I = P \cdot \frac{r_{bps}}{10000} \tag{3}$$

where P is principal and r_{bps} is the interest rate in basis points. Repayment updates the debt balance, adds events to the immutable history, and contributes to value factor restoration.

7 Liquidation & Arbitrage

7.1 Liquidation Triggers

When debt obligations become unserviceable according to protocol rules, automatic liquidation is triggered. The debt instrument transitions to liquidating status and the debt escalates through the liquidation system.

7.2 Liquidation Pools

Liquidation pools are structured with a single provider instrument for aggregate accounting and fungible pool shares for depositors. Key mechanics:

- Depositors receive proportional shares representing claims on pool assets
- Interest from lending activities distributes proportionally to shareholders
- Pools provide liquidity for debt repayment during liquidation
- All pool accounting is optimized for block-relevant validation

7.3 Arbitrage Mechanism

When pool liquidity is insufficient or for efficiency, debt is assigned to arbitrageurs as temporary instruments. Arbitrage rules are strictly enforced:

- **Profit retention**: If the arbitrageur repays the full debt from arbitrage proceeds, they retain remaining profits
- **Loss absorption**: If the arbitrageur fails to recoup the full debt amount, they assume full liability for that debt
- **Asset liquidation**: Failed arbitrageurs must liquidate remaining arbitrage assets to repay the debt
- **Liability limitation**: Arbitrageurs only assume liability upon failure to recoup, never for debts not initially assigned

The escalation path follows: automatic liquidation → liquidation pool → arbitrageur assignment.

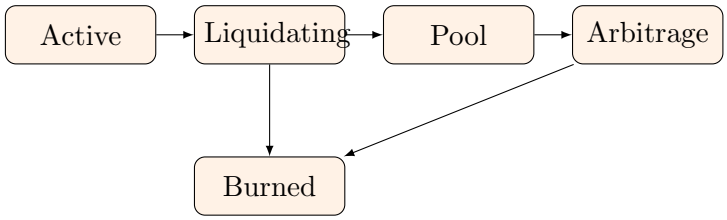


Figure 3: Liquidation escalation and burn paths

8 Debt Instruments & Trading

8.1 Transfer Restrictions

Debt instruments are non-transferable in the standard sense and cannot be treated as conventional digital collectibles. Trading is permitted only under strict conditions enforced at the protocol level.

8.2 One-Transaction Settlement Requirement

When transferring a debt instrument, the receiver must pay all outstanding debts plus accrued interest in a single transaction. This requirement is enforced atomically:

1. Transfer request is evaluated
2. If outstanding debt exists ($D_{total} > 0$), receiver must provide full payment in same transaction
3. If full payment not provided, dealing currencies noted on the instrument are permanently burned
4. Instrument is marked burned and becomes permanently unusable
5. If full payment provided, debts are satisfied and transfer proceeds

8.3 Loanability Reset on Successful Transfer

When a transfer succeeds and ownership actually changes (`from` $\neq$ `to`), the protocol resets the instrument's deposit-derived loanability to its initial value before recording the new owner:

1. `deposit_loanability` is reset to deposit_amount $\cdot$ 70/100.
2. `total_minted_from_nft` is set to the new `deposit_loanability`.
3. `loan_count` is reset to 0.
4. `update_value_factor()` recomputes the value factor from the new denominator.

A `LoanabilityReset` event is appended to the instrument's immutable `debt_history`, recording the transaction id, previous owner, new owner, previous loanability, new loanability, and block height. The reset preserves `available_collateral_backing`, `deposit_amount`, and repayment history. No reset occurs for self-transfers (`from` = `to`), for transfers with unpaid debt (the instrument is burned), or for already-burned instruments.

8.4 Burning Mechanism

Burning is permanent and irreversible. When triggered:

- All dealing currencies noted on the instrument are destroyed
- The instrument status becomes burned
- The instrument cannot be used for any economic activity
- Burn events are recorded in immutable history
- Burned currencies are not recoverable or redistributed

8.5 Profit Distribution on Sale

If a loan has been paid in full with interest, special profit distribution rules apply upon instrument sale:

- The seller receives the paid-off loaned currency amounts plus interest
- The other mint holder receives a more valuable instrument due to improved value factor
- These effects are reflected in the value factor mechanics

9 Value Factor Mechanics

9.1 Definition

The value factor V represents the economic value of a debt instrument relative to its obligations, determined by the ratio between available collateral backing and total minted currencies:

$$V_t = \frac{C_{avail}(t)}{M_{total}(t)} \cdot \rho(t) \tag{4}$$

where C_{avail} is available collateral backing, M_{total} is total currencies minted from the instrument, and $\rho(t)$ represents repayment restoration effects.

9.2 Behavior

- **Minting effect**: As more currencies are minted from an instrument, M_{total} increases relative to C_{avail}, causing V to decrease
- **Repayment restoration**: When currencies are repaid, C_{avail} increases and the repayment credit ρ restores value
- **Full repayment**: Paying off loans in full with interest affects value distribution on subsequent sales
- **Tracking**: Value factor is tracked on-chain and all changes are recorded in immutable history
- **Loanability reset on sale**: on a successful ownership change, `total_minted_from_nft` is reset to the initial loanability and V is recomputed

The value factor is bounded such that $0 \leq V \leq 1$ under normal conditions, with restoration mechanisms preventing permanent value collapse through repayment.

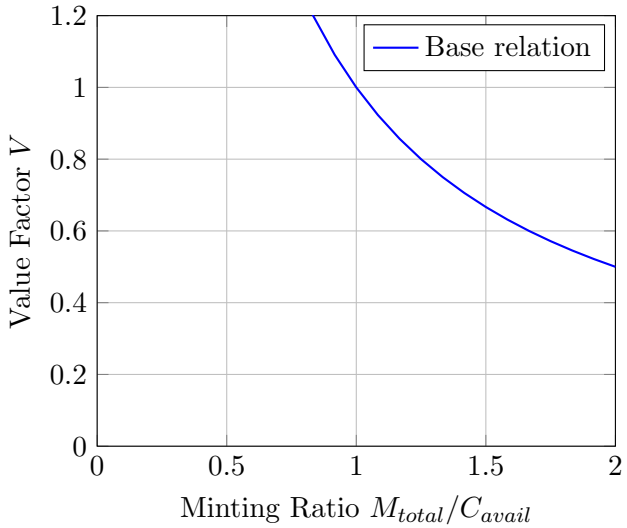


Figure 4: Value factor decreases as minting ratio increases, restored through repayment

10 Depositor Withdrawal

10.1 Overview

A depositor may withdraw the principal and accrued interest from a debt instrument before all associated loans are settled. The withdrawal mints the total withdrawn amount to the depositor in the instrument's loaned currency and triggers adjustments to affected loans, tracking of withdrawn funds, and, when loans remain unpaid, bounty creation.

10.2 Withdrawal Execution

Upon withdrawal, the protocol:

1. Verifies the instrument is active and has not already been withdrawn.
2. Mints principal plus accrued interest to the depositor.
3. Records a withdrawal entry with affected loan references.
4. Marks the instrument as having executed a withdrawal.
5. Tracks the withdrawn funds as held.

10.3 Affected Loan Adjustment

All loans affected by the withdrawal have their obligations discounted to forty percent of the original obligation. The discounted obligation remains payable by the loaner, while the instrument's debt history records each adjustment. This discount reflects the reduced collateral backing after withdrawal.

10.4 Withdrawn Funds Tracking and Loss

Withdrawn funds are tracked by withdrawal and may transition through held, transferred, lost, settled, or recovered states. If all tracked funds for a withdrawal are lost:

- Loaners who paid at the start receive refunds of their paid amounts.
- Affected loan obligations are released or marked refunded.
- Adjusted obligations are set to zero.

This ensures that lost withdrawn value does not remain an unbounded liability against loaners.

10.5 Bounty Hook

If unpaid affected loans remain after withdrawal, a bounty is created from the withdrawn value, as described in the Bounty System section. The bounty auctions the withdrawn value and either settles it to a bidder or distributes and burns it.

11 Yield Splitting for Fully Repaid Instruments

11.1 Overview

When a borrower fully repays a debt instrument with no prior repayments on that instrument, the borrower becomes a *yield splitter* for the instrument. Yield splitting diverts a portion of the interest that would otherwise accrue to depositors to the splitter, converted one-to-one to PAI, and accrues it as splitter earnings. The mechanism rewards disciplined full repayment while preserving depositor returns through a governed, recurring distribution. Yield splitting is tracked immutably on the instrument through dedicated debt events.

11.2 Qualifying Repayment

A repayment qualifies when all of the following hold before the repayment is applied:

- The instrument exists and is not burned, paid off, or frozen.
- The outstanding debt before repayment is strictly greater than zero.
- The repayment amount is at least the outstanding debt before repayment.
- The instrument's repayment count is zero.

The no-prior-repayment condition ensures that only the first full repayment on an instrument creates a yield splitter; subsequent repayments by the same or other borrowers do not re-qualify.

11.3 Yield Splitter Addition and Initial PAI Payment

On a qualifying repayment:

1. The borrower is added to the instrument's `yield_splitters` list if not already present, and a `YieldSplitterAdded` event is recorded.
2. The loaned amount is reconstructed as $L = d + c$, where d is the debt before repayment and c is the cumulative repaid amount before this repayment.
3. The interest basis is $I = \text{calculate_interest}(L)$ using the instrument's interest rate in basis points.
4. The split amount is $S = I \cdot p/10000$, where p is `yield_split_percentage_bps`, defaulting to 1000 (ten percent).
5. S is credited to the borrower's PAI balance and to PAI supply.
6. The instrument's `yield_splitter_pending` entry for the borrower is increased by S.
7. The instrument's `deposit_interest_accrued` is reduced by S (saturating).
8. A `YieldSplitPaid` event is recorded.

The repayment itself still applies the existing full-payoff value boost, cumulative repayment tracking, repayment count increment, and taker-score bonus. The split is therefore additive to, not a replacement for, the existing full-repayment rewards.

11.4 Recurring Distribution

After the repayment count is incremented, if the new count is a multiple of six and yield splitting has been established, a recurring distribution occurs. For each splitter in `yield_splitters`:

1. The pending earnings E for that splitter are retrieved (defaulting to zero).
2. The share is $D = E \cdot 10/100$ (ten percent).
3. If $D > 0$: D is transferred from the splitter's PAI balance to the original depositor's PAI balance, a `YieldSplitDistribution` event is recorded, and the splitter receives +8 taker score.
4. Regardless of D, the splitter's pending earnings are reset to zero.

The recurring distribution is an internal PAI transfer between accounts; PAI supply is unchanged. The taker-score adjustment uses the existing clamped $0 \ldots 100$ helper and keeps both `taker_score` and `taker_scores` in sync. The six-repayment cadence and ten-percent share are fixed by protocol constants.

11.5 Freeze Enforcement

While an instrument is frozen for yield-split governance, no transaction may touch it. Specifically, `Repay`, `NftTransfer`, `DepositorWithdraw`, and `Liquidate` operations return `NftFrozen` if the target instrument exists and is frozen, before any state mutation. This guarantees that governance completes without concurrent economic activity on the instrument.

11.6 Depositor Freeze and Minimum Splitter Count

The original depositor may freeze an active, non-burned instrument by submitting a `YieldSplitFreeze` transaction. The instrument must not already be frozen and the submitter must be the depositor. Freezing sets `frozen = true` and records a `YieldSplitFrozen` event.

While frozen, the depositor may adjust the minimum required number of yield splitters via `SetMinimumYieldSplitters`. The minimum defaults to six and is stored as `minimum_yield_splitter_users`. Adjusting it records a `MinimumYieldSplittersSet` event.

11.7 Yield Splitter Voting and Establishment

Eligible yield splitters are those listed in `yield_splitters` whose taker score is at least one. Each eligible splitter may vote on a proposed split percentage in basis points via `YieldSplitVote`. Votes are recorded in `yield_votes` keyed by voter, and a `YieldSplitPercentageVoted` event is recorded.

Establishment occurs only when both of the following hold:

- The number of distinct votes is a strict majority of `yield_splitters`: votes$\times 2 >$ splitters.
- The number of yield splitters is at least the configured minimum: splitters $\geq$ minimum_yield_splitter_users

When these conditions are met, proposals are tallied. The proposal with the most votes wins; ties are broken by the highest percentage. The winning percentage becomes `yield_split_percentage_bps`, `yield_split_established` becomes true, `frozen` becomes false, and a `YieldSplitEstablished` event is recorded. The instrument is thereby unfrozen and recurring distributions may begin.

11.8 Data Model

The instrument gains the following fields, all with serde defaults for backward compatibility with previously serialized instruments:

- `yield_splitters`: vector of splitter addresses (default empty).
- `minimum_yield_splitter_users`: minimum splitter count (default six).
- `frozen`: governance freeze flag (default false).
- `yield_split_established`: whether recurring distribution is active (default false).
- `yield_split_percentage_bps`: split percentage in basis points (default 1000).
- `yield_votes`: map from voter to proposed percentage.
- `yield_splitter_pending`: map from splitter to accrued earnings.

The instrument also exposes `is_yield_splitter(address)`.

New debt events: `YieldSplitterAdded`, `YieldSplitFrozen`, `MinimumYieldSplittersSet`, `YieldSplitPercentageVoted`, `YieldSplitEstablished`, `YieldSplitPaid`, and `YieldSplitDistribution`.

New transactions: `YieldSplitFreeze`, `SetMinimumYieldSplitters`, and `YieldSplitVote`. Block-relevant state construction touches the depositor or voter account and the instrument for each. A new error, `NftFrozen`, signals that an operation was rejected because the instrument is frozen.

11.9 Protocol Constants

- `YIELD_SPLIT_DEFAULT_PERCENTAGE_BPS` = 1000 (ten percent).
- `YIELD_SPLIT_RECURRING_PERCENT` = 10 (ten percent of accrued earnings).
- `YIELD_SPLIT_RECURRING_INTERVAL` = 6 repayments.
- `YIELD_SPLIT_TAKER_DELTA` = +8 taker score per recurring distribution.
- `YIELD_SPLITTER_MIN_TAKER_SCORE` = 1 for voter eligibility.

12 Bounty System

12.1 Overview

A bounty is created when a depositor withdraws from a debt instrument while affected loans remain unpaid. The bounty represents the withdrawn minted value and is auctioned on-chain in the instrument's loaned currency. If a bidder wins, the winner receives the bounty currency and pays PAI to the reward wallet. If no bids arrive by expiry, a portion of the value is distributed as PAI to all wallets and the remainder is burned over time.

12.2 Bounty Creation

Upon depositor withdrawal, if unpaid affected loans exist, a bounty is created with:

- **Source instrument**: the originating debt instrument.
- **Reward wallet**: defaults to the original depositor.
- **Currency**: the instrument's loaned currency.
- **Original amount**: the withdrawn minted total, equal to principal plus accrued interest.
- **Loan count**: the number of affected unpaid loans.
- **Consistency score**: the fraction of affected loans that were repaid at the start of the withdrawal adjustment.
- **Average interest**: the mean interest implied by the affected loans.
- **Time limit**: a block horizon derived from the amount and consistency.
- **End height**: creation height plus the time limit.

A matching auction is created with the same end height. The bounty status begins as open and transitions to auctioning as bids are placed.

12.3 Loanable Reduction and Payment Schedules

Loan issuance scales the loanable portion of deposits with the number of loans already associated with the instrument:

$$\text{loanable_pct} = \text{clamp}(70 - 5 \cdot \text{loan_count},\ 20,\ 70) \tag{5}$$

Payment schedules are stored on loans as a due height computed from the total possible loans, the interest rate in basis points, and the deposit amount:

$$\text{payment_due_height} = \text{borrow_height} + f(\text{total_possible_loans},\ \text{interest_rate_bps},\ \text{deposit_amount}) \tag{6}$$

12.4 Consistency Scoring and Time Limits

The consistency score measures repayment discipline among affected loans:

$$\text{consistency_score} = \frac{\#\{\text{affected loans repaid at start}\}}{\#\{\text{affected loans}\}} \tag{7}$$

The bounty time limit is reduced for more consistent repayment:

$$\text{time_limit} = \max(\text{base_blocks} - \lfloor 50 \cdot \text{consistency_score} \rfloor, 5) \tag{8}$$

where base_blocks is a protocol parameter. The average interest is computed across affected loans and used both for settlement payments and for the timed burn rate.

12.5 On-Chain Auction and Bidding

Bounties are auctioned with the highest bid winning at expiry. A bid is accepted only if:

1. the bounty is open or auctioning,
2. the current height is before the end height,
3. the bid amount exceeds the current highest bid, and
4. the bidder has sufficient balance.

Each accepted bid is recorded with the bidder, amount, and height, and becomes the new highest bid. The bounty tracks the current winner and winning bid.

12.6 Expiry Processing

At the block whose height equals a bounty's end height, the bounty is resolved:

- **Highest bid exists**: the winner receives the current bounty amount in the bounty currency; the winner pays PAI equal to interest plus the average interest to the reward wallet; the status becomes settled.
- **No bids**: twenty percent of the original amount is distributed as PAI equally among all wallets in ledger state; the remainder becomes `burn_remaining`; the status becomes expired-without-bids and timed burning begins.

12.7 Timed Burn

For each expired bounty with remaining value, every block burns:

$$\text{burn_amount} = \min(\text{burn_per_block}, \text{burn_remaining}) \tag{9}$$

where burn_per_block equals the average interest. The amount is removed from the bounty currency supply and subtracted from `burn_remaining`. When `burn_remaining` reaches zero, the bounty status becomes burned.

12.8 Economic Invariants

- Bounty amount equals the withdrawn minted total at creation.
- Auctions close exactly at their end height.
- Settlement transfers bounty currency to the winner and PAI to the reward wallet.
- No-bid distribution allocates exactly twenty percent of the original amount.
- Timed burning never exceeds the remaining amount and removes value only from supply.

13 Bridging

PAI supports external asset deposits through a bridging mechanism. The initial implementation uses a federated bridge approach:

- **Federation**: A trusted federation of validators signs bridge transfers for external asset deposits and withdrawals
- **Asset representation**: External assets are represented on-chain with full protocol integration
- **Deposit flow**: Bridge transfers create debt instruments as specified by protocol rules
- **Evolution path**: The design allows migration to a more decentralized mechanism in the future

The bridge enforces signature thresholds from the federation before confirming transfers into the protocol.

14 PAI Economic Model

14.1 Native Currency

PAI serves as the native currency of the protocol, used for bids in the consensus auction, fees, bonds, and rewards to network participants.

14.2 Supply Mechanics

PAI supply is dynamic per block. For each block h, new supply is minted according to:

$$S_{mint}(h) = R_{fixed} + F_{priority}(h) \tag{10}$$

where R_{fixed} is the fixed block reward and $F_{priority}$ represents priority fees collected. The total supply changes block by block as specified by protocol parameters.

14.3 Fee Market

PAI implements a fee market similar to EIP-1559:

- **Base fee**: Adjusts per block based on block fullness relative to target gas usage
- **Base fee burning**: The base fee is permanently burned, removed from circulation
- **Priority fees**: Tips paid by users go to block producers

Base fee adjustment follows:

$$f_{base}(h+1) = f_{base}(h) \cdot \left(1 + \frac{g(h) - g_{target}}{g_{target} \cdot d}\right) \tag{11}$$

where $g(h)$ is gas used in block h, g_{target} is target gas, and d is the denominator controlling adjustment rate.

14.4 Reward Distribution

Block rewards and fees are distributed to network participants:

- Paired block producer receives portion of rewards and priority fees
- Paired validator receives portion of rewards
- Committee members receive rewards for attestation participation
- Base fees are burned (not distributed)

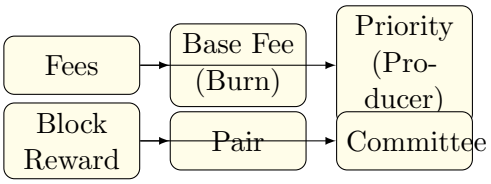


Figure 5: PAI fee and reward distribution

15 Validation, Attestation & Cross-Verification

15.1 Block-Relevant State Construction

For each block, nodes construct a view containing only touched elements:

- Touched accounts
- Touched assets
- Touched debt instruments

Cryptographic commitments over these sets enable efficient verification without scanning full history.

15.2 Attestation Process

The paired validator validates all transactions against the block-relevant view and produces an attestation containing block identifier, validator identity, validity status, and cryptographic signature.

15.3 Cross-Verification

The block producer receives the attestation and verifies it against its own view of the shared replicated ledger. Both parties reference identical ledger state, simplifying cross-verification. Disagreements trigger slashing mechanisms.

16 Reputation & Slashing

16.1 Reputation System

Reputation determines committee selection weight and reflects participant reliability. Factors include:

- **Uptime**: Node availability and responsiveness

- **Success rate**: Successful attestations and valid participation
- **Recency decay**: Recent behavior weighted more heavily with time-based decay
- **Stake consideration**: Bonded stake influences reputation weighting

Reputation decays over time according to:

$$R_{t+1} = R_t \cdot \delta + \Delta_t, \quad \delta \approx 0.99 \tag{12}$$

where Δ_t represents reputation changes from observed behavior.

16.2 Slashing

Slashing penalties are scaled by severity:

Severity	Penalty	Consequence
Minor	Partial bond slash	Temporary penalty
Major	Substantial bond slash	Significant penalty
Critical	Full bond slash	Permanent removal from bonded set

Table 1: Slashing severity levels

Slashable offenses include:

1. Block producer accepting or verifying false validator claims against the shared replicated ledger
2. Validator producing false attestations

16.3 Committee Selection

Committee members are selected via reputation-weighted selection from non-paired eligible participants, favoring high-reputation, reliable nodes while maintaining decentralization.

17 Security Considerations

17.1 Economic Security

- Bonding requirements ensure economic stake in honest behavior
- Slashing creates cost for malicious actions
- Auction mechanism prevents predictable block production
- Committee provides additional oversight against pair collusion

17.2 Debt Enforcement

- Immutable history prevents debt record tampering
- One-transaction settlement prevents partial debt evasion
- Automatic burning enforces consequences for failed settlements
- Liquidation escalation provides multiple recovery paths

17.3 Consensus Security

- Shared replicated ledger enables cross-verification
- Pairwise attestation with committee quorum prevents unilateral actions
- Sealed-bid auction prevents front-running of selection
- Reputation system incentivizes long-term honest participation

17.4 Invariant Enforcement

Critical invariants maintained by the protocol:

- Debt history remains immutable
- Burned instruments and currencies are permanently destroyed
- Full debt payment required for instrument transfer or burn triggered
- Slashing conditions are enforced objectively
- Block-relevant validation maintains consistency with shared ledger

18 Conclusion

PAI introduces a fundamentally different approach to blockchain design by embedding debt as a native primitive. The combination of debt instruments, strict transfer mechanics, liquidation mechanisms, and custom consensus creates a system purpose-built for debt-based economic activity. The shared replicated ledger architecture with block-relevant validation balances security and efficiency, while the economic model aligns incentives through bonding, reputation, slashing, and dynamic fee mechanisms. The protocol provides a complete framework for debt creation, enforcement, and resolution without relying on layered smart contract systems for core debt logic.

19 References

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Buterin, V. et al. (2015). Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.
3. Buterin, V. (2019). Ethereum Improvement Proposal 1559: Fee Market Change for ETH 1.0 Chain.
4. Wood, G. (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger.

20 Appendix

20.1 Key Parameters

Parameter	Value
Committee Size	4
Minor Slash	10% of bond
Major Slash	50% of bond
Critical Slash	100% of bond
Default Interest Rate	5% (500 bps)
Liquidation Threshold	80% (8000 bps)
Reputation Decay Factor	0.99
Target Gas Usage	Protocol defined

Table 2: Core protocol parameters

20.2 Glossary

Debt Instrument A protocol-level record tracking collateral, debt obligations, and history for a deposit.

Deposit Loanability Deposit-derived loanable amount recorded on a debt instrument, reset to 70% of the deposit on a successful sale.

Loanability Reset Immutable `LoanabilityReset` event recording the reset of deposit loanability and loan count on a successful ownership change.

Block-Relevant Validation Validation limited to state elements touched by transactions in the current block.

Custom Attested Bundle Consensus mechanism combining sealed-bid auctions, pairwise production, and committee attestation.

Shared Replicated Ledger Canonical ledger state maintained identically across all network participants.

Value Factor Ratio-based metric representing instrument value relative to minted currencies and collateral.

Liquidation Pool Aggregate liquidity pool with provider instrument and fungible shares for debt resolution.

Bounty Auctioned value created when a depositor withdraws while loans remain unpaid.

Consistency Score Fraction of affected loans repaid at withdrawal, used to shorten bounty time limits.